





4 · 12

Sekiro · ·

500

· Kiro

() , Sekiro

: SDK App

: .md · .md · SukiSU_Ultra .md

-
- ,
-
- A:Sekiro RPC ()
- B:
- C: + ADB
- D: ()
-
- 7
- Demo
-
-

- A
- B

I

1.1

```
[  ]
App
  SDK      80+
  POST
    deviceToken
    deviceToken
```

```
[  ]
Python/Go

"      "
POST
```

, " "

1.2

	¥1500 + ¥30/	¥0(IP+)
	1 1	(IP)
	,	,
	SIM	
	/	

1.3

" , " ' ' App"

" " ,

1.4

, :

1. (KeyMint / Widevine L1) — / , App ,
2. — App APK SHA256 , " "
3. - — 5 " , " "
- (D)

8 , :

2.1

ID	SDK deviceToken	deviceToken ,
		,
		, SDK
App		, " "
	IP	IP
-		,
	TrickyStore	

2.2

, :

1:deviceToken

deviceToken , , :- (A D) - SDK
(B) - SDK (C)

2:

80+ , " "

3: " "

SDK , 5-10 " " , , " SDK
"



A. Sekiro RPC	,	¥10/	(			
B.	SDK,	¥1/				+
C.	Redroid/	¥30/				" "
D.	+ +	¥5/				

A:Sekiro RPC ()

4.1

Python ()

-
- JSON
- IP
-
- deviceToken / sign :
- HTTP sekiro server

HTTP RPC

Sekiro Server(Java,)

-
-
-

[1] [2] [N]
App App App
Frida Frida Frida
sekiro-client /

4.2 Sekiro Server

```
# 1.      ( Linux )
git clone https://github.com/yint-tech/sekiro-open.git
cd sekiro-open

# 2.  Docker ( )
docker pull yintech/sekiro:latest
docker run -d --name sekiro \
  -p 5620:5620 \
  -p 5621:5621 \
  yintech/sekiro:latest

# 3.
curl http://your-server:5620/business-demo/list
#      {"data":[], "ok":true}
```

4.3 Frida Sekiro Client

(root + App +):

```
# 1.  frida-server
adb push frida-server-16.x.x-android-arm64 /data/local/tmp/
adb shell "su -c 'chmod 755 /data/local/tmp/frida-server'"
adb shell "su -c '/data/local/tmp/frida-server &'"

# 2.  sekiro-client.js +
adb push sekiro_yidun_xianyu.js /data/local/tmp/
```

sekiro_yidun_xianyu.js :

```

// 1.   sekiro client
function require_sekiro() {
  Java.perform(function () {
    var client = new SekiroClient(
      "ws://your-server:5621/business/register?" +
      "group=yidun-pool&" +
      "clientId=phone-001"
    );

    // === 1:   deviceToken ===
    client.registerAction("get_yidun_token", function (req, resp) {
      Java.perform(function () {
        try {
          var SmAntiFraud = Java.use("com.netease.mobsec.SmAntiFraud");
          var token = SmAntiFraud.getDeviceId();
          resp.success({ deviceToken: token, phoneId: "phone-001" });
        } catch (e) {
          resp.failed("getDeviceId error: " + e);
        }
      });
    });

    // === 2:   mtop sign ===
    client.registerAction("xianyu_mtop_sign", function (req, resp) {
      Java.perform(function () {
        try {
          var data = req.data;    //
          var apiName = req.apiName; //

          var SecGuardMgr = Java.use(
            "com.taobao.wireless.security.adapter.JNICLibrary"
          );
          // hook
          var sign = /* hook    sign */;
          resp.success({ sign: sign });
        } catch (e) {
          resp.failed("sign error: " + e);
        }
      });
    });
  });
}

```

```
// === 3: ( )===
client.registerAction("yidun_heartbeat", function (req, resp) {
  Java.perform(function () {
    // SDK
    var WatchMan = Java.use("com.netease.mobsec.rjsb.watchman.WatchMan");
    WatchMan.report.call();
    resp.success({ ok: true });
  });
});
}

setImmediate(require_sekiro);
```

:

```
frida -U -f com.taobao.idlefish -l sekiro_yidun_xianyu.js --no-pause
```

4.4 Python

```

import requests
import time
import random

SEKIRO_HOST = "http://your-server:5620"
GROUP = "yidun-pool"

def call_sekiro(action, **kwargs):
    """    sekiro    """
    url = f"{SEKIRO_HOST}/business/invoke"
    params = {"group": GROUP, "action": action, **kwargs}
    r = requests.post(url, params=params, timeout=10)
    j = r.json()
    if j.get("status") != 0:
        raise RuntimeError(f"sekiro {action} failed: {j}")
    return j["data"]

def get_yidun_token():
    return call_sekiro("get_yidun_token")["deviceToken"]

def xianyu_sign(api_name, data):
    return call_sekiro("xianyu_mtop_sign", apiName=api_name, data=data)["sign"]

def heartbeat():
    """    SDK    , 5-10    """
    return call_sekiro("yidun_heartbeat")

# ===    ===

def publish_xianyu_item(profile, account, item_data):
    api = "mtop.idle.publish"
    sign = xianyu_sign(api, item_data)
    token = get_yidun_token()

    headers = {
        "x-mtop-sign": sign,
        "x-device-id": token,
        "x-uid": account.uid,
        "user-agent": profile.ua,
        "x-app-version": profile.app_version,
        # ...
    }

```

```

r = requests.post(
    "https://acs.m.taobao.com/gw/" + api,
    headers=headers,
    json=item_data,
    proxies={"https": account.proxy},
    timeout=15
)
return r.json()

```

4.5

, :

```

# Sekiro server
class PhonePool:
    def __init__(self):
        self.phones = ["phone-001", "phone-002", ...]
        self.last_call = {} # phone -> timestamp

    def get_phone(self):
        #
        now = time.time()
        candidates = sorted(
            self.phones,
            key=lambda p: self.last_call.get(p, 0)
        )
        for p in candidates:
            if now - self.last_call.get(p, 0) > 1.5: # 1.5s
                self.last_call[p] = now
            return p
        time.sleep(1)
        return self.get_phone()

```

:1000~3000 / (App) 10 100

4.6 " "

, ():

```
# 1: N
adb shell "su -c 'sh /data/local/tmp/randomize_profile.sh com.taobao.idlefish'"
adb shell "su -c 'pm clear com.taobao.idlefish'"
adb shell "am start -n com.taobao.idlefish/.MainActivity"
sleep 30 # App

# 2: , ( )
```

I B:

5.1

, **SDK** :

1. SDK(libDeviceFingerprint.so)
2. :
 - (80+ protobuf/JSON)
 - (AES + RSA +)
 - (POST lookup.dun.163.com)
3. Python :
 - JSON
 - , POST
 - deviceToken
 - token

5.2

SDK	1-3	(VMP +)
80+	1	
(AES key)	1-2	
RSA	3-5	
Python	1	
+		
()	1-2	

5.3

:- , - -

:- , , - (VMP + + +) - " " (

), IP - - ()

5.4

- > 10
- 2
- ()
- " "

:GitHub , demo " API"

/ ,ROI

5.5 ()

, :

1. App APK
2. unzip + .so
lib/arm64-v8a/libNetEaseMobsec.so
3. IDA Pro Ghidra , :
- Java_xxx_collect ()
- Java_xxx_encrypt ()
- Java_xxx_report ()
4. Frida hook ,dump
5. ,
6. : AES key , RSA
7. Python , cryptography
8. , Python

C: + ADB

6.1

[: Redroid/ / /]
App +

ADB shell input + Accessibility Service

SDK ,

6.2

	¥40/	(ROM)	()
	¥30/		
	¥35/		
	¥20/		
	¥25/		
Redroid	¥80/ (GPU)	(AOSP)	-

6.3 Redroid

```
# 1. GPU (NVIDIA T4/A10 )
# 2. Docker + nvidia-container-toolkit
# 3. redroid
docker pull redroid/redroid:14.0.0_64only-latest

# 4. ( )
docker run -d --rm --privileged \
  --name redroid-001 \
  --network=host \
  -v /data/redroid-001:/data \
  redroid/redroid:14.0.0_64only-latest \
  androidboot.redroid_gpu_mode=guest

# 5. ADB
adb connect localhost:5555
```

6.4 AOSP ()

Redroid :

```
target: frameworks/base
1. android.os.Build SystemProperties /data/local/profile.json
2. android.provider.Settings$Secure ANDROID_ID
3. android.telephony.TelephonyManager
4. WifiInfo MAC/BSSID
5. AOSP, redroid
```

redroid , profile.json , " "

6.5

: - - - ,

: - (ROM) - Redroid AOSP () - GPU ,5

redroid 1 - , " "

D: ()

7.1

" " " "

:

- (,)
- / ()
- 80%
- sekiro token " "

(/ /)

- Sekiro (10~50)
- deviceToken / sign
-
-

- Redroid ()
-
-

7.2

— (80%)

/		
		, " "
/		

— Sekiro (15%)

			,
/			

— Redroid (5%)

	Redroid
	Redroid
	Redroid

7.3

5 , :

Week 1: Sekiro +
- 1 sekiro server
- 5 KernelSU + Frida
- "Python "
Week 2:
- 5 profile
- profile
- " "

Week 3:
- / /
- IP
-

Week 4:
- 1
-
- (10)

Week 5+:
- 50 100 500



8.1 Profile JSON


```
{
  "_meta": {
    "profile_id": "profile-001",
    "model_template": "redmi-k70",
    "created_at": 1729400000,
    "use_count": 0,
    "last_used_at": null,
    "bound_account": null,
    "status": "fresh"
  },

  "build": {
    "model": "Redmi K70",
    "brand": "Redmi",
    "manufacturer": "Xiaomi",
    "product": "rothko",
    "device": "rothko",
    "board": "rothko",
    "hardware": "qcom",
    "fingerprint": "Redmi/rothko/rothko:14/UKQ1.230804.001/V816.0.7.0:user/release-keys",
    "host": "pangu-build-component-system-2080xxxx",
    "user": "builder",
    "tags": "release-keys",
    "type": "user",
    "display": "UKQ1.230804.001",
    "id": "UKQ1.230804.001",
    "bootloader": "unknown",
    "radio": "MOLY.LR12A.R3.MP.V90.1.P10",
    "incremental": "V816.0.7.0.UMNCNXM"
  },

  "version": {
    "sdk_int": 34,
    "release": "14",
    "codename": "REL",
    "security_patch": "2024-08-01",
    "incremental": "V816.0.7.0.UMNCNXM"
  },

  "screen": {
    "width": 2712,
```

```
"height": 1220,
"dpi": 460,
"xdpi": 442.5,
"ydpi": 443.345,
"density": 2.875,
"refresh_rate": 120
},

"cpu": {
  "abi": ["arm64-v8a", "armeabi-v7a", "armeabi"],
  "model": "Snapdragon 8 Gen 2",
  "cores": 8,
  "max_freq_khz": 3187200,
  "hardware": "qcom"
},

"memory": {
  "ram_total_bytes": 12884901888,
  "rom_total_bytes": 274877906944
},

"sensors": [
  {"name": "ICM4X6XX Accelerometer", "vendor": "InvenSense", "type": 1, "version": 1},
  {"name": "ICM4X6XX Gyroscope", "vendor": "InvenSense", "type": 4, "version": 1},
  {"name": "AK0991X Magnetometer", "vendor": "AKM", "type": 2, "version": 1},
  {"name": "BMP380 Barometer", "vendor": "Bosch", "type": 6, "version": 1},
  {"name": "STK3338 Light", "vendor": "Sensortek", "type": 5, "version": 1},
  {"name": "STK3338 Proximity", "vendor": "Sensortek", "type": 8, "version": 1}
],

"ids": {
  "imei": "865432109876543",
  "imei2": "865432109876544",
  "meid": "A0000012345678",
  "imsi": "460017012345678",
  "android_id": "a1b2c3d4e5f60718",
  "serial": "RKQ7K23M5KQ12345",
  "oaid": "8a8a8a8a-1234-5678-9abc-def012345678",
  "vaid": "v1a2b3c4-1234-5678-9abc-def012345678",
  "aaaid": "a1b2c3d4-1234-5678-9abc-def012345678",
  "widevine_id": "ce7c0033-fb12-4cce-9eaa-1f6c5a8b1234",
  "gsf_id": "2a8c0123456789ab",
```

```
"huawei_id": null
},

"network": {
  "mac_wifi": "02:00:aa:bb:cc:dd",
  "mac_bt": "02:00:11:22:33:44",
  "wifi_bssid": "a4:5e:60:11:22:33",
  "wifi_ssid": "TP-LINK_A1B2",
  "wifi_signal": -55,
  "scan_results": [
    {"bssid": "a4:5e:60:11:22:33", "ssid": "TP-LINK_A1B2", "level": -55},
    {"bssid": "b4:5e:60:11:22:34", "ssid": "ChinaNet-AAA", "level": -68}
  ],
  "operator": "46001",
  "operator_name": " ",
  "country_iso": "cn",
  "phone_country": "CN"
},

"locale": {
  "language": "zh",
  "country": "CN",
  "timezone": "Asia/Shanghai",
  "timezone_offset_min": 480,
  "fonts_count": 87
},

"battery": {
  "level": 78,
  "scale": 100,
  "temperature": 285,
  "voltage": 4123,
  "status": "discharging",
  "health": "good"
},

"time": {
  "boot_timestamp_ms": 1729380000000,
  "elapsed_realtime_ms": 7200000,
  "build_timestamp_ms": 1693497600000
},
```

```

"app": {
  "package_name": "com.taobao.idlefish",
  "version_name": "7.16.5",
  "version_code": 71605,
  "first_install_time": 1729400000000,
  "last_update_time": 1729400000000,
  "signature_md5": "abc123def456...",
  "installer_package": "com.android.vending"
},

"ip_binding": {
  "proxy": "socks5://user:pass@proxy.example.com:1080",
  "expected_region": " - ",
  "expected_isp": " "
},

"behavior": {
  "open_count_today": 0,
  "last_open_at": 0,
  "last_active_minute": 0,
  "next_action_after_ms": 1500
}
}

```

8.2 (!)

, , " "

```

: model = "Redmi K70"

:
- brand      = "Redmi"
- manufacturer = "Xiaomi"
- product    = "rothko"(K70    )
- fingerprint = "rothko"
- screen     = 2712x1220 / 460dpi
- cpu        = "Snapdragon 8 Gen 2" 8
- ram        [12GB, 16GB, 24GB]
- rom        [256GB, 512GB, 1TB]
- sensors     (ICM4X6XX )
- security_patch K70    (2023-12)
- sdk_int     34
-           zh_CN(    )
- timezone    Asia/Shanghai

```

8.3 (" ")

```

boot_time      = NOW - random(2 , 3 )    # 0,
app_install_time = NOW - random(1 , 90 )    # 0,
app_first_install = app_install_time - 0~7    #
build_time     = K70    + random(0, 30 )    #
last_active    = NOW - random(1 , 30 )    # "    "

```

8.4 IP- -

```

profile.network.operator = "46001" (    )

profile.ip_binding.proxy      IP :
-      IP
-      ( IP )
-      GPS/WiFi BSSID

```

```

:
```

```
# profile ,
target_region = " " - "
target_isp = " "

#
proxy = proxy_pool.get_by(region=target_region, isp=target_isp)

# IP
ip_info = requests.get("https://ip.example/api?ip=...").json()
assert ip_info["province"] == " "
assert " " in ip_info["isp"]
```

8.5 " "

, :

A. ()

```
# DevCheck Pro / AIDA64 / Device Info HW
# JSON
# sensor / oaid / widevine_id
```

:1 1 20-30

B. GitHub : - android device fingerprint dataset - android sensor list collection - mobile device characteristics database

,

C. " " / ,500-2000 (),

: 30 +

8.6 Profile

```
class ProfilePool:
    def acquire(self, account_id):
        """    fresh profile    """
        p = self.db.find_one({"status": "fresh"})
        p["status"] = "in_use"
        p["bound_account"] = account_id
        p["last_used_at"] = time.time()
        self.db.update(p)
        return p

    def release(self, profile_id, success):
        if success:
            p = self.db.get(profile_id)
            p["use_count"] += 1
            if p["use_count"] >= 3: # 3
                p["status"] = "retired"
            self.db.update(p)
        else:
            #
            self.db.update_status(profile_id, "burned")
```

:profile , , , profile

7

9.1

- device profile
- profile
- profile /

9.2 IP

- profile
- IP profile /
- IP 1-3

9.3

```
      :  
T+0:  
T+5min: 30  
T+10min: 1  
T+30min: App  
T+8h: , 5  
T+1d:  
T+3d:  
T+5d:  
T+7d:
```

, 24 ,

9.4 " "

```
: boot_time = 0  
: app_install_time = current_time  
: profile boot_time  
  
: boot_time = NOW - random(2h, 72h)  
: profile
```

9.5

- = profile ,
- profile ()

- = IP , IP

9.6

SDK , :

```
async def heartbeat_loop(account):
    while account.online:
        await asyncio.sleep(random.randint(300, 600))
        await call_yidun_heartbeat(account)
```

9.7

- 1-2 ,
 -
 - 2
-

10.1

```
xianyu-protocol/  
  config/  
    settings.py  
    machines.json      #  
  core/  
    profile_pool.py    #  
    account_pool.py    #  
    proxy_pool.py      #  
    sekiro_client.py   # sekiro  
  protocol/  
    xianyu_api.py      #  
    headers_builder.py #  
    sign_helper.py     #  
  behavior/  
    scheduler.py        #  
    timing.py           #  
    heartbeat.py        #  
  monitor/  
    metrics.py          #  
    alarm.py            #  
  main.py               #
```

10.2 sekiro_client.py

```
import requests
from typing import Any, Dict
import logging
import time

logger = logging.getLogger(__name__)

class SekiroClient:
    def __init__(self, host: str, group: str, timeout: int = 10):
        self.host = host.rstrip("/")
        self.group = group
        self.timeout = timeout

    def invoke(self, action: str, **kwargs) -> Dict[str, Any]:
        url = f"{self.host}/business/invoke"
        params = {"group": self.group, "action": action, **kwargs}

        for attempt in range(3):
            try:
                r = requests.post(url, params=params, timeout=self.timeout)
                j = r.json()
                if j.get("status") == 0:
                    return j["data"]
                logger.warning(f"sekiro {action} returned non-zero: {j}")
            except Exception as e:
                logger.warning(f"sekiro {action} attempt {attempt}: {e}")
                time.sleep(0.5 * (attempt + 1))
        raise RuntimeError(f"sekiro {action} failed after 3 attempts")

    def get_yidun_token(self) -> str:
        return self.invoke("get_yidun_token")["deviceToken"]

    def xianyu_sign(self, api_name: str, data: str) -> str:
        return self.invoke("xianyu_mtop_sign", apiName=api_name, data=data)["sign"]

    def heartbeat(self) -> bool:
        return self.invoke("yidun_heartbeat").get("ok", False)
```

10.3 profile_pool.py

```

import json
import random
from pathlib import Path
from typing import Optional, Dict
import threading
import time

class ProfilePool:
    def __init__(self, db_path: Path):
        self.db_path = db_path
        self.lock = threading.Lock()
        self._load()

    def _load(self):
        with open(self.db_path, "r", encoding="utf-8") as f:
            self.profiles = json.load(f)

    def _save(self):
        with open(self.db_path, "w", encoding="utf-8") as f:
            json.dump(self.profiles, f, ensure_ascii=False, indent=2)

    def acquire(self, account_id: str, region: str = None) -> Optional[Dict]:
        with self.lock:
            for p in self.profiles:
                if p["_meta"]["status"] != "fresh":
                    continue
                if region and p.get("ip_binding", {}).get("expected_region") != region:
                    continue
                p["_meta"]["status"] = "in_use"
                p["_meta"]["bound_account"] = account_id
                p["_meta"]["last_used_at"] = int(time.time())
                self._save()
                return p
            return None

    def burn(self, profile_id: str):
        """ profile ( ) """
        with self.lock:
            for p in self.profiles:
                if p["_meta"]["profile_id"] == profile_id:
                    p["_meta"]["status"] = "burned"

```

```
        self._save()
        return

def release(self, profile_id: str, success: bool = True):
    with self.lock:
        for p in self.profiles:
            if p["_meta"]["profile_id"] == profile_id:
                if success:
                    p["_meta"]["use_count"] = p["_meta"].get("use_count", 0) + 1
                    if p["_meta"]["use_count"] >= 3:
                        p["_meta"]["status"] = "retired"
                    else:
                        p["_meta"]["status"] = "fresh"
                else:
                    p["_meta"]["status"] = "burned"
            self._save()
        return
```

10.4 headers_builder.py

```
def build_xianyu_headers(profile: dict, account: dict, sekiro: SekiroClient) -> dict:
    """
    profile
    ( )"""

    token = sekiro.get_yidun_token()

    return {
        "user-agent": (
            f"AliApp(IF/{profile['app']['version_name']}) "
            f"WindVane/8.5.0 "
            f"{profile['build']['manufacturer']}/{profile['build']['model']} "
            f"Android/{profile['version']['release']}"
        ),
        "x-app-version": profile["app"]["version_name"],
        "x-device-id": token,
        "x-uid": account["uid"],
        "x-sid": account["sid"],
        "x-utdid": profile["ids"]["serial"][:24], # utdid serial
        "x-imei": profile["ids"]["imei"],
        "x-mac": profile["network"]["mac_wifi"].replace(":", ""),
        "x-os": "android",
        "x-os-version": profile["version"]["release"],
        "x-screen": f"{profile['screen']['width']}x{profile['screen']['height']}",
        "accept-language": "zh-CN,zh;q=0.9",
        "x-network": "WIFI",
        "x-bssid": profile["network"]["wifi_bssid"],
        "content-type": "application/x-www-form-urlencoded",
    }
```

10.5 timing.py —

```
import random
import time
from datetime import datetime, time as dt_time

#
HOUR_WEIGHT = {
    0: 0.1, 1: 0.05, 2: 0.02, 3: 0.01, 4: 0.01, 5: 0.02,
    6: 0.1, 7: 0.3, 8: 0.5, 9: 0.7, 10: 0.8, 11: 0.85,
    12: 0.9, 13: 0.85, 14: 0.8, 15: 0.85, 16: 0.85, 17: 0.8,
    18: 0.9, 19: 0.95, 20: 1.0, 21: 1.0, 22: 0.85, 23: 0.5
}

def should_act_now() -> bool:
    """
    """
    h = datetime.now().hour
    return random.random() < HOUR_WEIGHT[h]

def random_action_gap() -> float:
    """
    ( )"""
    # 80% 5-30 ,20% 1-10 ( " ")
    if random.random() < 0.8:
        return random.uniform(5, 30)
    return random.uniform(60, 600)

def random_session_duration() -> int:
    """
    ( )"""
    # 3-10 ,
    return int(random.gauss(300, 120))
```


10.6

```
def publish_item_flow(account_id: str, item_data: dict):
    pool = ProfilePool(Path("data/profiles.json"))
    sekiro = SekiroClient("http://sekiro:5620", "yidun-pool")

    # 1.    fresh profile
    profile = pool.acquire(account_id, region="  - ")
    if not profile:
        raise RuntimeError("no fresh profile available")

    profile_id = profile["_meta"]["profile_id"]

    try:
        # 2.    :
        browse_homepage(profile, account_id, sekiro)
        time.sleep(random_action_gap())

        browse_my_items(profile, account_id, sekiro)
        time.sleep(random_action_gap())

        # 3.    (    ,    )
        enter_publish_page(profile, account_id, sekiro)
        time.sleep(random.uniform(30, 90)) #

        # 4.
        result = call_publish_api(profile, account_id, sekiro, item_data)

        # 5.
        sekiro.heartbeat()

        pool.release(profile_id, success=True)
        return result

    except Exception as e:
        # 6.    profile
        pool.burn(profile_id)
        raise
```

11.1

profile	> 30%/	
IP	< 80%	
sekiro	> 5s	
QPS	> 5/s	()
	> 2%/	
deviceToken	> 5%	()

11.2

L1	profile	burned,	profile
L2	profile	2	sekiro
L3	IP	3	profile IP 30
L4	5	3	token +
L5	< 50%	30	,

11.3

```
class IPBlacklist:
    def __init__(self):
        self.banned = {} # ip -> unban_at

    def ban(self, ip, minutes=30):
        self.banned[ip] = time.time() + minutes * 60

    def is_banned(self, ip):
        t = self.banned.get(ip)
        return t and t > time.time()
```

I

12.1 deviceToken 401

```
: token (imei/utdid)
: profile ID ,
```

12.2 profile

```
: , " "
: " "
```

12.3

```
: , — profile
:- " " profile(boot_time , ) - 24 - / " "
```

12.4 sekiro

```
:- - SDK " " -
```

: - CPU/ - QPS < 0.5/s - (DevCheck)

12.5

: " "

: " "

12.6 token, N

:

: :

```
pm clear com.taobao.idlefish
rm -rf /sdcard/.um /sdcard/.mobsec ...
am start ... # 30 SDK
```

12.7 IP ,

: SIM IMSI

: IMSI 5 IP

```
46000 -
46001 -
46003 -
46011 -
```

12.8 redroid

: , ANDROID_ID

: profile.json, AOSP framework

I A

A.1 /

yint-tech/sekiro-open	RPC	,
CreditTone/hooker	RPC	
frida/frida		
lexiforest/curl_cffi	TLS	TLS
bogdanfinn/tls-client	TLS	Go

A.2

gzu-liyujiang/Android_CN_OAID	OAID	OAID
NoDev-Fate/StarHook	LSPosed	
Magisk-Modules-Repo/MagiskHidePropsConf	Magisk	Build props

A.3

LSPosed/LSPosed	Xposed	Hook
5ec1cff/TrickyStore	Magisk	Play Integrity
chiteroman/PlayIntegrityFix	Magisk	Play Integrity
Dr-TSNG/Hide-My-AppList	LSPosed	App

A.4

remote-android/redroid-doc	Redroid
google/android-cuttlefish	Google
waydroid/waydroid	Linux

A.5 IP

jhao104/proxy_pool	
snail007/goproxy	

B

B.1

- **mitmproxy** + CA (root)
- **HTTPCanary / reqable** ()
- **JustTrustMe / TrustMeAlready** SSL Pinning

B.2

```
https://lookup.dun.163.com/v3/d/i #
https://lookup.dun.163.com/v3/d/h #
https://crash.dun.163.com/... #
https://config.dun.163.com/... # SDK
```

B.3 ()

```
POST /v3/d/i HTTP/1.1
Host: lookup.dun.163.com
User-Agent: yd-sdk/1.x
Content-Type: application/x-www-form-urlencoded
```

```
d=<base64(AES (JSON ))>
&v=<sdk_version>
&s=< >
&k=<RSA (AES key)>
&sn=< , HMAC>
```

JSON : - imei / android_id / oaid / mac / serial / build.* / 80+ - key
,RSA SDK

B

1. :
 2. D()
 3. 30 ,
 IP

:

- 5~15 (Sekiro)
- 1 sekiro server
- 1 Python
- 1 IP (IP)
- 1 device profile (+)
- 1

:

- 5 50~100
- 15 300~500

SukiSU_Ultra

- Kiro
- 2026